

Detecting and Mitigating OWASP Top 10 Threats

Design, Implementation, and Security Evaluation of a Database-Driven Web Application

Zubair Idris Aweda · Student ID: 2306535

BSc (Hons) Computer Science · University of Wolverhampton

FINAL PRESENTATION & PROJECT DEFENSE

Project Introduction & Rationale

A persistent gap exists between theoretical cybersecurity guidance and real-world developer implementation. This project addresses that gap through a novel A/B testing architecture.

Version A — Vulnerable

Deliberately engineered with severe OWASP Top 10 security design flaws for controlled exploitation testing.

Version B — Hardened

A parallel, fully secured counterpart implementing industry-standard defensive patterns and mitigations.

OWASP SecBank: A dual-nature financial web application built to empirically test, measure, and contrast developer security realities.



Research Question, Aims & Goals

Core Research Question

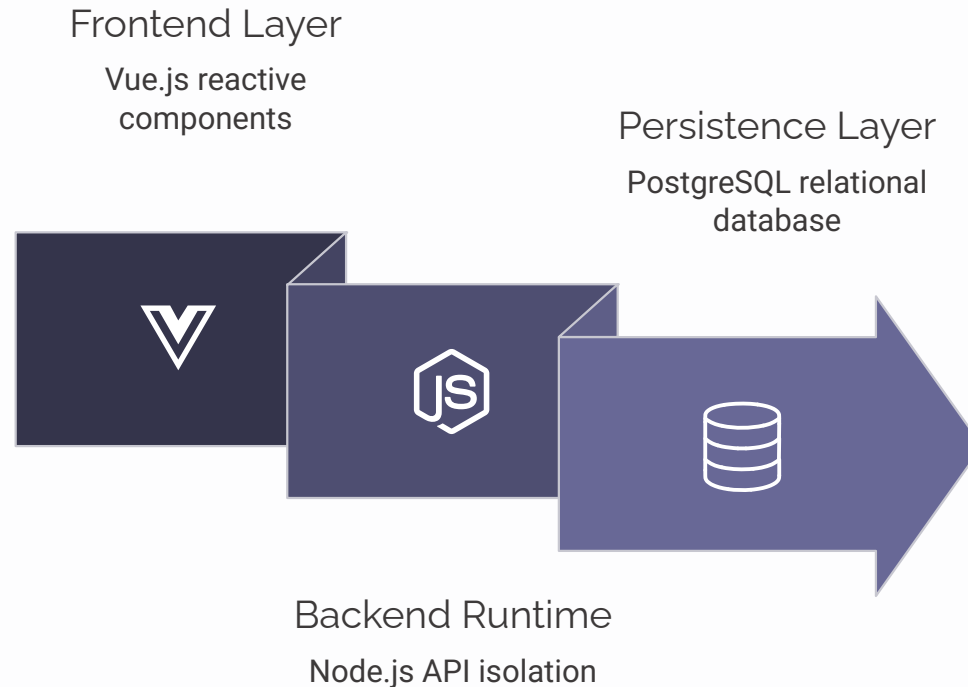
"How can OWASP Top 10 threats be best managed in modern database-driven web applications?"

Primary Aim: Evaluate and contrast the visibility of OWASP vulnerabilities between automated defensive tooling and structured manual security processes.

Core Project Goals

- Construct a fully functional, relationally backed online financial environment
- Map 10 critical security exploits directly into application source code
- Establish a comparative benchmark: tool efficacy vs. manual intuition

Engineering Approach & Architecture



A decoupled, modular architecture was selected to accurately implement both parameter-driven bugs and deep relational injection flaws – while maintaining clean separation between vulnerable and hardened code paths.

Artefact Demonstration: Validating Vulnerabilities

This section provides a quick demonstration of the OWASP SecBank application, focusing on four key test cases designed to expose and validate the intentionally engineered OWASP Top 10 vulnerabilities.

1

Test Case 1: SQL Injection

Demonstrate how malicious input in login forms can bypass authentication mechanisms.

2

Test Case 2: Security Misconfiguration

Expose sensitive information or functions due to improperly configured server settings or default credentials.

3

Test Case 3: Cryptographic Failure

Illustrate hard-coded secrets and unencrypted data being written to database.

4

Test Case 4: Insecure Design

Show how injected different versions handle bad

A01: Broken Access Control

Vulnerable Code (Version A)

```
const userId = req.params.id;  
const query = `SELECT * FROM accounts WHERE user_id  
= ${userId}`;  
const result = await pool.query(query);
```

Hardened Code (Version B)

```
// req.user comes from verified JWT in authMiddleware  
if (req.user.userId !== req.params.id && req.user.role  
!== 'admin') {  
  return res.status(403).json({ message: 'Forbidden' });  
}  
const result = await pool.query(  
  'SELECT * FROM accounts WHERE user_id = $1',  
  [req.params.id]  
);
```

Vulnerability: URL parameters (`req.params.id`) enabled arbitrary cross-user object access (IDOR) — relies on direct user input values without verifying authorization context on the server side.

Fix: UUIDs replaced integers; server-side token parsing enforced.

Detection & Analysis

SAST Tools: Missed entirely. Static analyzers treat this as syntactically correct queries.

DAST Tools: Missed. Automated active crawlers lack authorization context.

Manual Testing: Successfully found by swapping account IDs via parameter manipulation.

A02: Cryptographic Failures

Vulnerable Code (Version A)

```
const JWT_SECRET = 'super_secret_key_123';

// Registration stores plaintext:
const query = `
  INSERT INTO users (username, password, full_name,
    email)
    VALUES ('${username}', '${password}', '${full_name}',
    '${email}')
    RETURNING *;
`;
```

Hardened Code (Version B)

```
const JWT_SECRET = process.env.JWT_SECRET;
// Server refuses to start if absent

// Registration hashes first:
const hash = await bcrypt.hash(password, 12);
await pool.query(
  'INSERT INTO users (username, password_hash,
    full_name, email) VALUES ($1,$2,$3,$4)',
  [username, hash, full_name, email]
);
```

Vulnerability: Credentials were kept in unhashed, plain text, while server cryptographic secrets were committed directly to the core codebase:

Fix: Passwords are salted and hashed using bcrypt with an explicit computation cost factor of \$12\$. Keys are systematically loaded from runtime system variables using process environment matrices.

Detection & Analysis

SonarQube (SAST): Triggered a direct high-severity alert. Flagged the hardcoded secret as an absolute system Blocker.

Snyk (SCA): Identified explicit entropy issues in static code blocks.

Manual Audit: Database inspection immediately exposed plaintext credential strings.

A03: Injection

Vulnerable Code (VERSION A)

```
const query = `
  SELECT * FROM users
  WHERE username = '${username}'
  AND password = '${password}'
`;
await pool.query(query);

// Also in logout:
const query = `DELETE FROM tokens WHERE token =
'${token}'`;
```

Hardened Code (VERSION B)

```
const result = await pool.query(
  'SELECT * FROM users WHERE username = $1',
  [username]
);
// Password checked separately with bcrypt, never in
SQL
await bcrypt.compare(password,
result.rows[0].password_hash);

// Logout:
await pool.query('DELETE FROM tokens WHERE token =
$1', [token]);
```

Vulnerability: Input streams are interpolated into live statements directly, letting users breakout of execution fields: unsanitized string concatenation in live database queries enables SQL injection attacks.

Fix: Parameterized queries (\$1 placeholders) throughout the application.

Detection & Analysis

SAST Tools: Flagged instantly, highlighting unparameterized user variable interpolation points.

OWASP ZAP (DAST): Identified injection boundaries via error-based payload probes.

Manual Pen-Test: Confirmed full data extraction via classic ' OR '1'='1 authentication bypass strings.

A04: Insecure Design

Vulnerable Code (VERSION A)

```
const currentBalance =  
parseFloat(checkResult.rows[0].balance);  
const transferAmount = parseFloat(amount);  
  
if (currentBalance < transferAmount) {  
  return res.status(400).json({ message: 'Insufficient  
funds' });  
}  
// No check that transferAmount > 0 — negative values  
pass through
```

Hardened Code (VERSION B)

```
const transferAmount = parseFloat(amount);  
  
if (transferAmount <= 0) {  
  return res.status(400).json({ message: 'Invalid transfer  
amount' });  
}  
  
const currentBalance =  
parseFloat(checkResult.rows[0].balance);  
if (currentBalance < transferAmount) {  
  return res.status(400).json({ message: 'Insufficient  
funds' });  
}
```

Vulnerability: The api transaction routing logic accepted any raw numeric format type without business-layer validation. While positive amounts deducted funds correctly, sending a negative amount balance input (-£1000) mathematically increased user credit states without throwing errors. This is a fundamental design flaw rather than a typical broken code syntax issue.

Fix: Integrated explicit object schema contracts using the Joi validation middleware library right at the router layer: Enforced strict value validation rules (`Joi.number().positive().required()`).

Detection & Analysis: Completely hidden from every automated tool. Discovered purely via manual functional exploratory boundary edge case penetration test loops.

A05: Security Misconfiguration

Vulnerable Code (VERSION A)

```
try {  
  const result = await pool.query(query);  
  res.status(201).json({ message: 'User registered', user:  
    result.rows[0] });  
} catch (error) {  
  res.status(500).json({ error: error.message, detail:  
    error });  
  // Leaks: constraint name, table name, stack trace  
}
```

Hardened Code (VERSION B)

```
try {  
  const result = await pool.query(query);  
  res.status(201).json({ message: 'User registered' });  
  // Note: full user row NOT returned either  
} catch (error) {  
  next(error); // Handled by central error middleware  
  // Client only sees: { "error": "An unexpected error  
  occurred" }  
}
```

Vulnerability: CORS Controls: Set to wide-open wildcard configurations (Access-Control-Allow-Origin: *). Debug Leakage: Raw internal engine stack outputs were routed back to the client interface when queries failed.

Fix: Configured explicit origin array maps limiting cross-origin calls to trusted domain layers. Implemented global catch-all exception blocks to return sanitized, generic error IDs to public clients.

Detection & Analysis: OWASP ZAP (DAST): Highly effective. Flagged permissive cross-origin headers, missing anti-clickjacking parameters, and raw internal error disclosure leaks instantly.

Ao6: Vulnerable Components

Vulnerable Code (VERSION A)

```
"node-serialize": "^0.0.4"
```

Hardened Code (VERSION B)

```
// node-serialize removed entirely  
// Cookie parsing uses native JSON.parse() — no RCE risk
```

Vulnerability: Intentionally declared an older, unpatched dependency in the ecosystem manifest (node-serialize@0.0.4). This library variant contains a well-documented flaw where untrusted object properties can trigger arbitrary remote code execution via immediate functional execution blocks.

Fix: Removed the outdated third-party dependency tree and refactored object ingestion logic to rely safely on native, type-restricted `JSON.parse()` methods instead.

Detection & Analysis: Automatically detected by Snyk (SCA), which cross-referenced the version constraint string directly against active CVE data.

A07: Authentication Failures

Vulnerable Code (VERSION A)

```
const token = jwt.sign(  
  { userId: user.user_id, role: user.role },  
  JWT_SECRET,  
  { expiresIn: '30d' }  
);  
res.json({ message: 'Login successful', user:  
  result.rows[0] });  
// result.rows[0] = full DB row including password field
```

Hardened Code (VERSION B)

```
const token = jwt.sign(  
  { userId: user.user_id, role: user.role },  
  process.env.JWT_SECRET,  
  { expiresIn: '1h' }  
);  
res.json({  
  message: 'Login successful',  
  user: {  
    user_id: user.user_id,  
    username: user.username,  
    full_name: user.full_name,  
    email: user.email,  
    role: user.role  
    // password_hash intentionally excluded  
  }  
});
```

Vulnerability: JSON Web Tokens were configured to last for an excessive 30-day window, creating a massive exposure loop. The authentication payload sent back entire internal record models, including the active password hash string.

Fix: Reduced JWT lifespan strictly to 60 minutes. Enforced specific Data Transfer Object (DTO) sanitization scopes to explicitly strip sensitive parameters before data egress.

Detection & Analysis: Discovered by monitoring raw HTTP payload responses directly inside the browser Network panel during manual testing.

Ao8: Data Integrity Failures

Vulnerable Code (VERSION A)

```
app.use((req, res, next) => {
  if (req.cookies.profile_pref) {
    try {
      const str = Buffer.from(req.cookies.profile_pref,
'base64').toString();
      const obj = serialize.unserialize(str); // RCE if
crafted payload
      req.userPrefs = obj;
    } catch (err) {
      console.error("Deserialization error:",
err.message);
    }
  }
  next();
});

// Cookie set with httpOnly: false — readable by JS
const serializedPref = serialize.serialize(preferences);
const base64Pref =
Buffer.from(serializedPref).toString('base64');
res.cookie('profile_pref', base64Pref, { maxAge: 900000,
httpOnly: false });
```

Hardened Code (VERSION B)

```
app.use((req, res, next) => {
  if (req.signedCookies.profile_pref) {
    try {
      const obj =
JSON.parse(req.signedCookies.profile_pref);
      // JSON.parse cannot execute code — no RCE
possible
      req.userPrefs = obj;
    } catch (err) {
      req.userPrefs = {};
    }
  }
  next();
});

// Cookie signed with COOKIE_SECRET, httpOnly: true
res.cookie('profile_pref', JSON.stringify(preferences), {
  maxAge: 900000,
  httpOnly: true,
  signed: true,
  sameSite: 'strict'
});
```

Vulnerability: Ingested user configuration payloads straight from cookie headers into object parser engines without validation. Unsigned client cookies were deserialized directly into execution space without integrity verification, allowing attackers to tamper with role and permission values.

Fix: Implemented explicit cryptographic signature validation across all runtime cookie payloads. Enforced strict `HttpOnly` cookie constraints to block DOM read access.

Detection & Analysis:

Manual Testing: Successfully leveraged base64-encoded serialized payload adjustments to trigger live execution responses on the backend tier. Automation: Missed by default DAST checks due to the custom data formatting of the state payload.

A09: Logging Failures

Vulnerable Code (VERSION A)

```
// There is no logging infrastructure whatsoever.  
// Auth events, transfers, errors — all silent.
```

Hardened Code (VERSION B)

```
// Winston with PostgresTransport — every security  
event logged:  
logger.info({  
  event: 'LOGIN_SUCCESS',  
  userId: user.user_id,  
  ip: req.ip,  
  timestamp: new Date().toISOString()  
});  
  
logger.warn({  
  event: 'LOGIN_FAILURE',  
  username,  
  ip: req.ip  
});  
// Also written to system_logs DB table via  
PostgresTransport
```

Vulnerability: Zero audit logging — all security events silent. Critical operations like data deletion executed without any logging or accountability trail. The application processed financial ledger balances and authorization states in complete silence. No internal audit capture paths existed—meaning malicious attacks, privilege bypasses, and structural application errors left zero trail inside the data center.

Fix: Configured systematic stream event log capture hooks using the Winston utility suite, routing audit outputs to an immutable `system_logs` database. Collected distinct metrics (actor context timestamps, error hashes, boundary events).

Detection & Analysis: Found by absence of data artifacts — no logs existed to trace who performed destructive operations or when they occurred. Verified by noting the complete lack of trace details left in system paths after running malicious exploit loops against Version A.

A10: SSRF

Vulnerable Code (VERSION A)

```
// Server fetches whatever URL the user supplies — no validation
const { url } = req.body;
const response = await axios.get(url);
// Attacker can probe: http://169.254.169.254/metadata
// http://localhost:5432 (your DB port)
```

Hardened Code (VERSION B)

```
function isSafeUrl(rawUrl) {
  const parsed = new URL(rawUrl);
  if (parsed.protocol !== 'https:') return false;

  const privateRanges = [
    '127.0.0.0/8', '10.0.0.0/8',
    '172.16.0.0/12', '192.168.0.0/16',
    '169.254.169.254/32'
  ];
  // Resolve hostname and check against private ranges
  return !isPrivateIP(parsed.hostname, privateRanges);
}

const { url } = req.body;
if (!isSafeUrl(url)) {
  return res.status(400).json({ message: 'Invalid or unsafe URL' });
}
const response = await axios.get(url);
```

Vulnerability: The platform profile feature let users submit an image link string for their user avatar, which the server backend immediately requested and downloaded without verification. Attackers could use this behavior to turn the server into a proxy, probing private cloud infrastructure loops (`http://127.0.0.1:8080/metrics`). Unchecked user-supplied URLs enabled arbitrary backend fetches, creating an SSRF condition that could reach internal services, cloud metadata endpoints, or other restricted network resources.

Fix: Implemented network input filters to drop private IP networks (RFC 1918 ranges like `10.x`, `192.168.x`). Enforced strict protocol constraints, allowing only `https:` endpoints.

Detection & Analysis: SonarQube flagged this instantly, matching the arbitrary client-supplied URL arguments directly to request execution sinks. Found by SonarQube sink rules and validated by tracing outbound requests for unsafe destinations. The review confirmed missing protocol validation, absent DNS verification, and no filtering of private network segments in Version A.

Evaluation of Diagnostic Toolkit

Tool	Strengths	Limitations
SonarQube (SAST)	Strong at syntactic code matches, pattern signatures, and raw data flows	Blind to missing logic checks and complex execution paths
OWASP ZAP (DAST)	Identifies infrastructure misconfigurations, header failures, and CORS weaknesses	No visibility into internal dependency tree configurations
Snyk (SCA)	Flawless automation matching third-party packages against CVE databases	Zero insight into custom business logic bugs

 **Key Finding:** Business logic flaws (A04, A09) were **completely invisible** to all automated tooling – discoverable only through manual auditing.

Answer to Academic Question

Core Research Question:

"How can OWASP Top 10 threats be best managed in modern database-driven web applications?"

Research and empirical testing of Sec-Bank reveal that no single tool or methodology is sufficient. Effective management requires a strict, continuous Layered Defense-in-Depth Strategy that bridges the automation gap through three critical pillars:



Embedded Security Automation

The Syntactic Gate: Mandatory integration of SAST (SonarQube) and SCA (Snyk) directly into CI/CD pre-commit hooks. Automatically flags and blocks standard syntactic and known module vulnerabilities.



Safe-by-Default Architecture

The Engineering Foundation: Shifting security to the design phase ("Shift-Left"). Enforcing non-negotiable structural safe boundaries inside code defaults (e.g., query parameterization, randomized tokens, signed session objects).



Context-Aware Human Ingress

The Logical Validator: Regular manual code reviews and targeted behavioral penetration testing. Vital for exposing architectural business logic bypasses and severe context-driven flaws where automated scanners have a blind spot.

Lessons Learned & Open Floor



Shift-Left Realization

Establishing safe code architecture early costs far less than patching vulnerabilities late in development cycles.



Automated Tooling Limits

Security scanners are powerful helpers – but they cannot replace human code auditing for business logic flaws.



Full-Stack Competency

Deep engineering insight gained balancing transactional runtime requirements against strict data parsing barriers.

Sec-Bank Security Testing Framework · BSc (Hons) Computer Science
Dissertation Defense

Open for discussion: Architecture, implementation parameters, mitigation strategies, and evaluation methodologies.

Questions & Discussion

We welcome your thoughts, insights, and questions on the Sec-Bank Security Testing Framework.

Thank you for your time and attention.